LangChain, Chunking, and RAG Systems

Section 1: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 2: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
        from langchain.text_splitter import RecursiveCharacterTextSplitter

        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=100
        )

        chunks = text_splitter.split_text(long_document)

        print("Total chunks:", len(chunks))


        from langchain.embeddings import HuggingFaceEmbeddings
        from langchain.vectorstores import FAISS

        embedding_model = HuggingFaceEmbeddings()

        vector_db = FAISS.from_texts(
            chunks,
            embedding_model
        )

        query = "What is chunk overlap?"

        docs = vector_db.similarity_search(query)

        print(docs[0].page_content)


        from langchain.chains import RetrievalQA
        from langchain.chat_models import ChatOpenAI

        qa_chain = RetrievalQA.from_chain_type(
            llm=ChatOpenAI(),
            retriever=vector_db.as_retriever()
        )

        response = qa_chain.run(
            "Explain recursive chunking."
        )

        print(response)
```

Section 3: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 4: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 5: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)
chunks = text_splitter.split_text(long_document)
```

```
print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 6: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
```

```
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 7: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 8: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 9: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```

Section 10: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
        from langchain.text_splitter import RecursiveCharacterTextSplitter

        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=100
        )

        chunks = text_splitter.split_text(long_document)

        print("Total chunks:", len(chunks))


        from langchain.embeddings import HuggingFaceEmbeddings
        from langchain.vectorstores import FAISS

        embedding_model = HuggingFaceEmbeddings()

        vector_db = FAISS.from_texts(
            chunks,
            embedding_model
        )

        query = "What is chunk overlap?"

        docs = vector_db.similarity_search(query)

        print(docs[0].page_content)


        from langchain.chains import RetrievalQA
        from langchain.chat_models import ChatOpenAI

        qa_chain = RetrievalQA.from_chain_type(
            llm=ChatOpenAI(),
            retriever=vector_db.as_retriever()
        )

        response = qa_chain.run(
            "Explain recursive chunking."
        )

        print(response)
```

Section 11: LangChain and RAG Pipelines

LangChain is an orchestration framework for building LLM-powered applications. It connects prompts, vector databases, memory systems, retrieval pipelines, and external APIs into a unified workflow.

Chunking is one of the most important concepts in Retrieval-Augmented Generation. Large documents are divided into smaller overlapping segments so embeddings can capture semantic meaning effectively.

Vector databases such as FAISS, Pinecone, ChromaDB, and Weaviate are used to store embeddings and perform similarity search operations efficiently.

When experimenting with chunking strategies, developers often compare fixed-size, recursive, semantic, and markdown-aware chunking approaches to maximize retrieval quality.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = text_splitter.split_text(long_document)

print("Total chunks:", len(chunks))


from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS

embedding_model = HuggingFaceEmbeddings()

vector_db = FAISS.from_texts(
    chunks,
    embedding_model
)

query = "What is chunk overlap?"

docs = vector_db.similarity_search(query)

print(docs[0].page_content)


from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=vector_db.as_retriever()
)

response = qa_chain.run(
    "Explain recursive chunking."
)

print(response)
```